

DEPARTAMENTO DE INFORMÁTICA

INF2102 — PROJETO FINAL DE PROGRAMAÇÃO

CBIR — Explorador de Espaço Vetorial para Imagens de Embarcações

Gabriel Ribeiro

Orientador:
[preencher]

Rio de Janeiro
Julho de 2026

1 Breve Descrição

O CBIR — **Explorador de Espaço Vetorial** é uma ferramenta para *recuperação de imagens por conteúdo* (*Content-Based Image Retrieval*) aplicada a recortes de embarcações. O programa indexa *embeddings* de imagem em um banco de dados vetorial e permite **explorar visualmente o espaço de representação**: projeta a galeria indexada em 2D/3D via PCA, aceita uma imagem de consulta, e mostra onde ela se posiciona em relação aos aglomerados (*clusters*) de cada classe — além de prever, por votação *K-Nearest Neighbors* (KNN) sobre os vizinhos recuperados, a que classe a imagem consultada pertenceria, com um grau de confiança.

Principais funções

Função	Descrição
Indexação	Extraí <i>embeddings</i> de recortes de imagem com um modelo selecionável e os armazena no Milvus.
Projeção	Reduz os <i>embeddings</i> da galeria a 2D/3D com PCA para visualização.
Consulta visual	Projeta uma imagem nova no <i>mesmo</i> espaço da galeria e destaca seus vizinhos mais próximos.
Predição de classe	Vota, por KNN, a classe da imagem consultada e reporta a confiança.
Snapshot reproduzível	Exporta/reconstrói uma coleção a partir de um cache de <i>embeddings</i> (sem GPU).

Tabela 1: Funções oferecidas pelo programa.

Usuários visados: pesquisadores e estudantes de *CBIR* / visão computacional que precisam **conferir visualmente** se a recuperação e a classificação de suas representações estão coerentes — em vez de confiar apenas em métricas agregadas.

Natureza do programa: ferramenta utilitária funcional (prova de conceito madura), servindo de base experimental para uma dissertação de mestrado sobre rotulagem automática por recuperação.

Ressalva de uso: o programa **não** treina modelos e **não** é um classificador de produção. A predição KNN é uma *heurística de rotulagem por recuperação*, cuja confiabilidade é justamente o objeto de estudo. Os *embeddings* vêm de um modelo genérico externo (OpenCLIP), não de um modelo especializado no domínio.

2 Visão de Projeto

Esta seção apresenta quatro cenários (dois positivos e dois negativos) que orientam a intenção do criador e a interpretação do usuário.

2.1 Cenário Positivo 1 — Conferir um agrupamento coerente

Marina, pesquisadora de CBIR, quer saber se um *embedding* genérico separa bem recortes de **Traineira de Rebocador**. Ela indexa a galeria de amostra, abre o explorador, escolhe a projeção 3D e vê quatro nuvens de cor razoavelmente distintas. Ela arrasta um recorte de **Rebocador** como consulta: o ponto vermelho cai **dentro** da nuvem de **Rebocador**, os 10 vizinhos exibidos são todos rebocadores, e o painel de predição mostra “**seria rotulada como Rebocador — 90% de confiança**”. Marina conclui, visualmente, que a representação é adequada para essa classe naquela faixa de tamanho.

Este cenário evoca as funções centrais: indexação, projeção, consulta e predição. Note que Marina não precisou de nenhuma métrica agregada — a resposta veio da posição do ponto e da concordância dos vizinhos.

2.2 Cenário Positivo 2 — Trocar de modelo com garantia de consistência

Pedro desconfia que *patches* menores capturariam melhor embarcações pequenas. Ele reindexa a mesma galeria com `openclip-vit-b-16` em uma coleção separada. Ao abrir o explorador e selecionar essa coleção, a interface exibe “**modelo de embedding: openclip-vit-b-16**” e garante que qualquer consulta será *embeddada* com esse mesmo modelo. Pedro compara as duas projeções lado a lado e decide qual modelo separa melhor as classes — sem risco de comparar vetores de espaços diferentes.

Este cenário evoca a troca de modelos e a **garantia de consistência**: a coleção “lembra” com qual modelo foi construída, e o sistema recusa misturar espaços de *embedding*.

2.3 Cenário Negativo 1 — Consulta com modelo incompatível

Ana tenta, via API, consultar uma coleção construída com `openclip-vit-b-32` forçando o modelo `openclip-vit-b-16`. O sistema **recusa** a operação com um erro `409 Conflict` e a mensagem: “*a coleção foi construída com openclip-vit-b-32, mas a consulta pediu openclip-vit-b-16; os embeddings seriam incomparáveis*”.

Este cenário ilustra uma limitação **conhecida e desejada**: distâncias entre vetores de modelos diferentes não têm significado. O programa prefere falhar de forma clara a devolver um resultado silenciosamente incorreto. Não há como contornar isso pela interface, e é intencional.

2.4 Cenário Negativo 2 — Recorte ambíguo mal classificado

João consulta com um recorte de *Traineira* pequeno e distante, capturado ao fundo de uma cena. O ponto de consulta cai na fronteira entre *Traineira* e *Navio de Carga Geral*, e a predição KNN retorna “**Navio de Carga Geral**” com alta confiança — um erro. Ao inspecionar os vizinhos exibidos, João percebe que todos são embarcações pequenas e distantes da mesma câmera: visualmente parecidas, apenas alguns *pixels*.

Este cenário expõe uma limitação diferente da anterior: para objetos muito pequenos, o *embedding* genérico captura mais o contexto de cena do que o objeto. O programa não esconde isso — ao contrário, a ferramenta **serve exatamente para tornar esse tipo de falha visível**, o que é um resultado de pesquisa, não um defeito de software.

3 Documentação Técnica do Projeto

3.1 Modelo de Arquitetura

O sistema é organizado em três camadas com dependência unidirecional (*frontend* → API → *backend*), mais os contratos de dados e a observabilidade compartilhados (Figura 1).

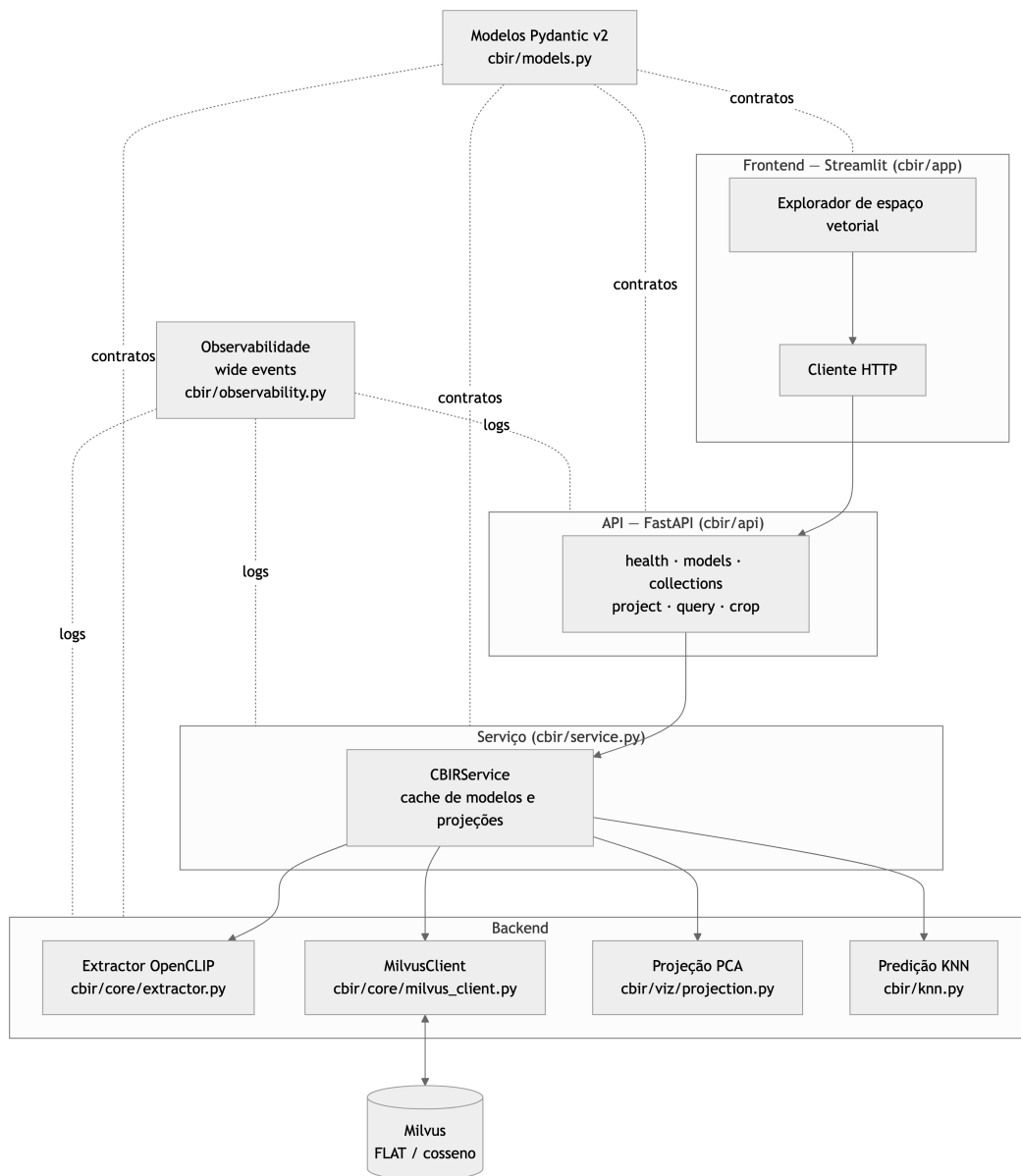


Figura 1: Arquitetura em três camadas. O *frontend* fala apenas com a API; a API delega ao serviço; o serviço orquestra o *backend* e o Milvus.

3.2 Modelo de Dados

A unidade canônica é o **recorte de bounding box**. Um *manifest* (JSONL, um registro por caixa) descreve cada item; o recorte é derivado em tempo de execução. Os *embeddings* e metadados são persistidos no Milvus. A Figura 2 apresenta as classes principais e a Figura 3 o diagrama entidade-relação das entidades persistidas.

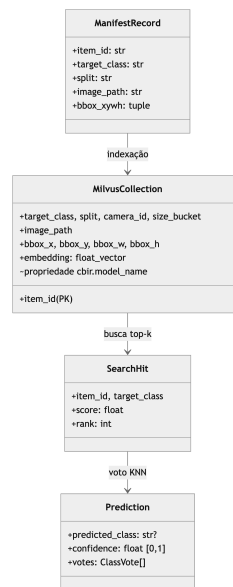
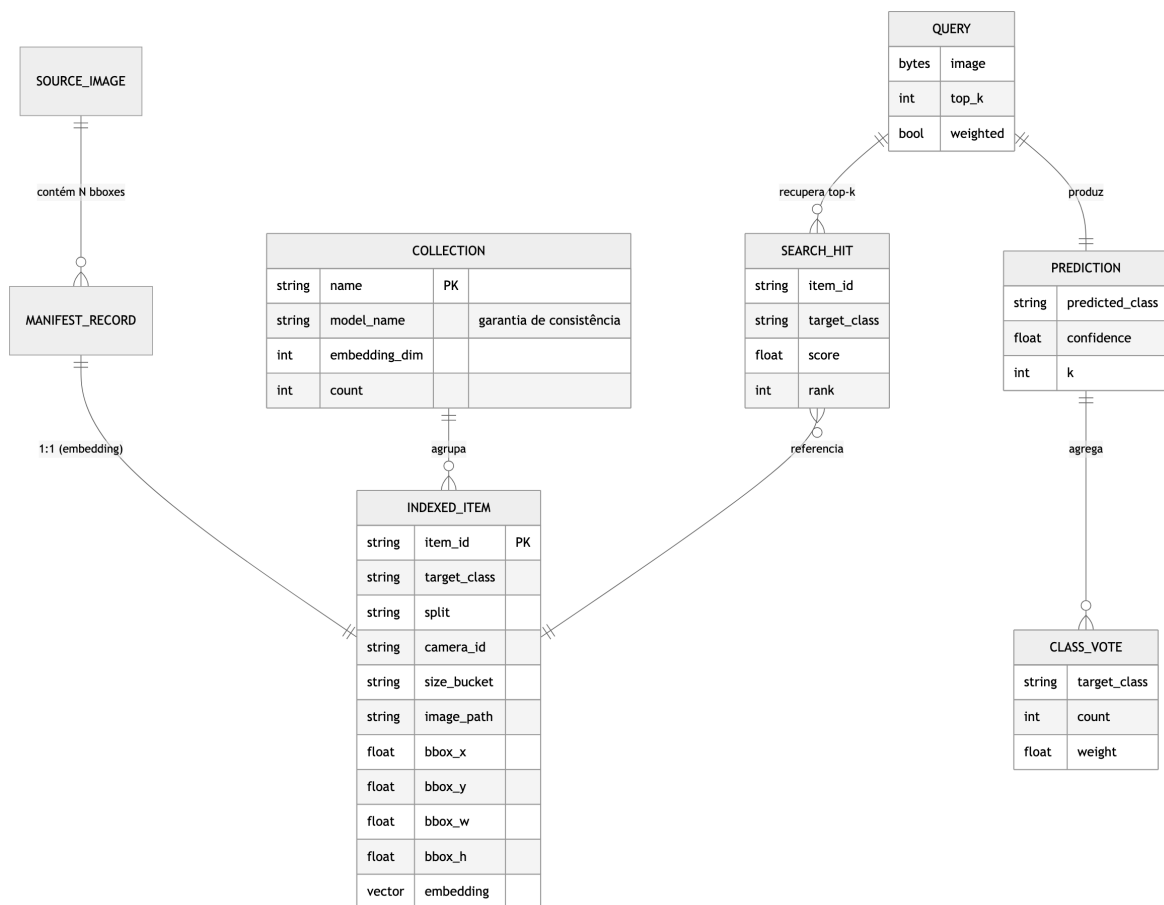


Figura 2: Principais classes de dados (contratos Pydantic v2).

Figura 3: Diagrama entidade-relação. Uma imagem-fonte contém N caixas; cada caixa vira um item indexado; cada consulta recupera k vizinhos, que alimentam uma predição.

O esquema de metadados carregado com cada vetor foi escolhido para ser exatamente o que o *frontend* precisa:

Campo	Uso
<code>target_class</code>	Cor do ponto no gráfico e voto KNN
<code>split</code> , <code>camera_id</code> , <code>size_bucket</code>	Facetas / <i>hover</i>
<code>image_path</code>	Servir o recorte como miniatura
<code>bbox_x..h</code>	Reconstruir o recorte quando necessário
<code>embedding</code>	Vetor para busca por cosseno
<code>propriedade</code>	Garantia de consistência de modelo
<code>cbir.model_name</code>	

Tabela 2: Esquema de metadados por vetor na coleção Milvus.

3.3 Fluxo de Indexação

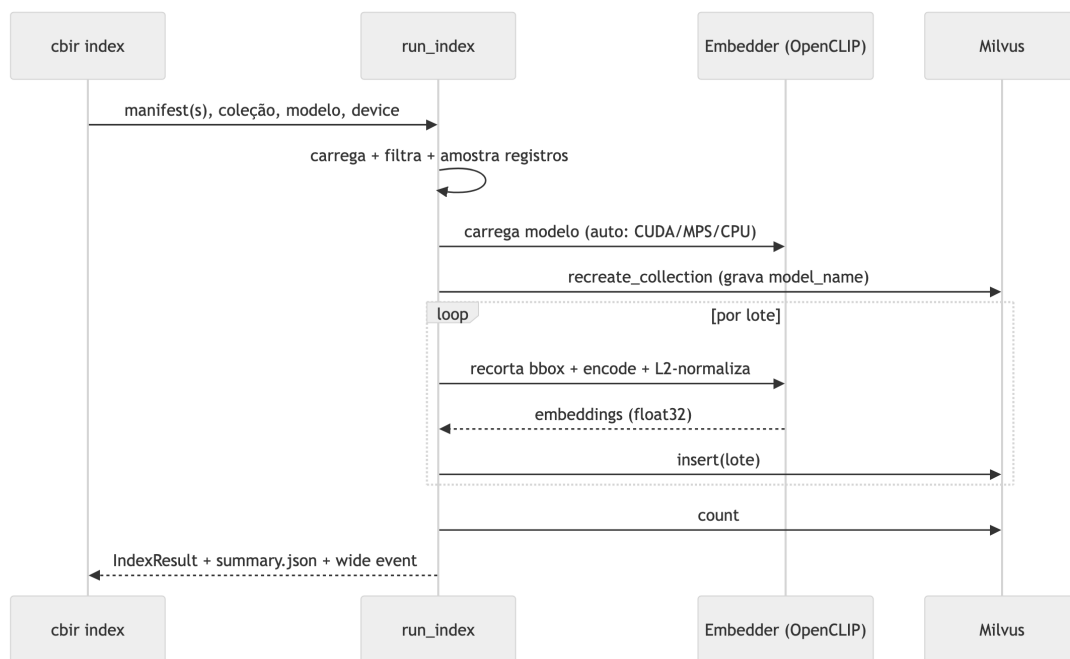


Figura 4: Fluxo de indexação: *manifest* → recorte → *embedding* → Milvus, com o modelo gravado na coleção.

3.4 Fluxo de Consulta

O fluxo de consulta (Figura 5) é o coração da ferramenta: embeda a imagem com o modelo da coleção, busca os vizinhos, projeta a consulta no mesmo PCA da galeria e vota a classe.

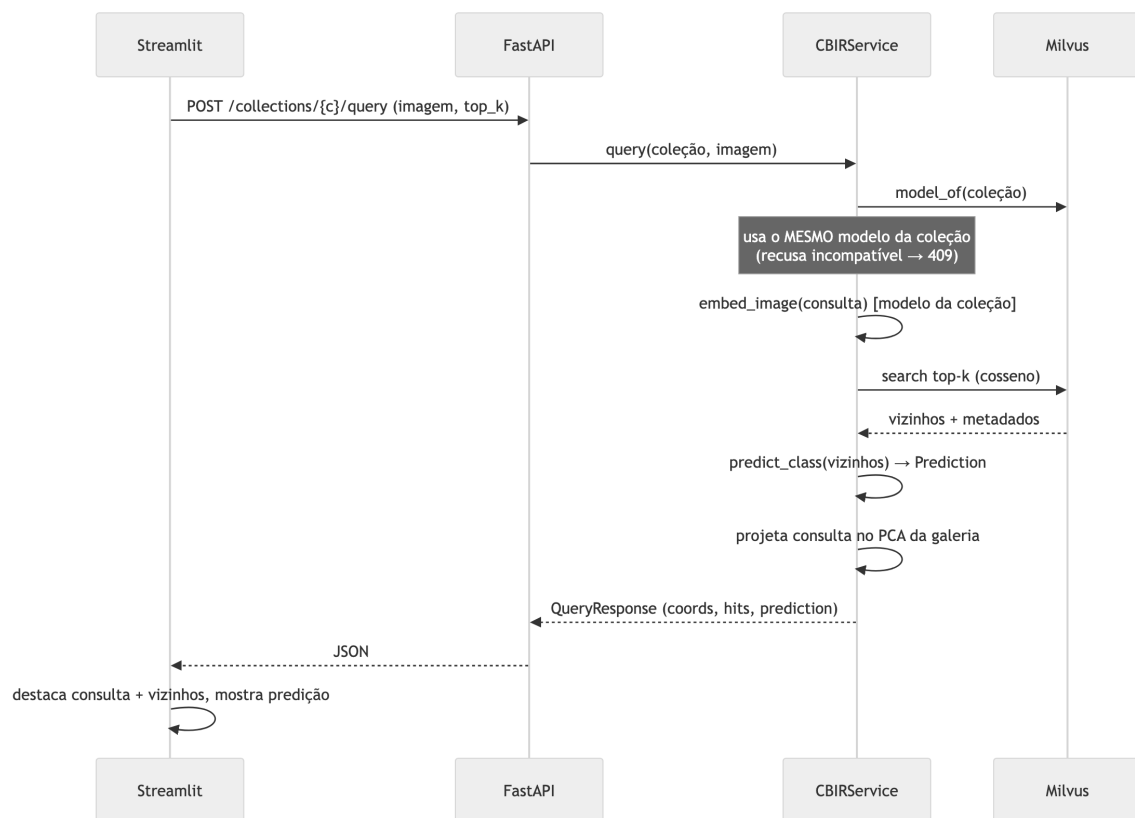


Figura 5: Fluxo de consulta. A consulta é sempre *embeddada* com o modelo da coleção; misturar espaços é recusado com 409.

3.5 Por que PCA (e não t-SNE/UMAP)

Critério	PCA	t-SNE / UMAP
Projetar consulta nova no mesmo espaço	Exato e barato	Sem transform exato
Determinismo	Sim	Estocástico
Preserva distâncias globais	Sim (linear)	Foco em estrutura local
“Onde minha consulta cai vs. clusters”	Ideal	Enganoso

Tabela 3: Justificativa da escolha de PCA para a projeção.

A escolha de PCA é o que torna correta a pergunta central do programa: *onde esta imagem nova cai em relação aos clusters existentes?* — algo que exige aplicar exatamente a mesma transformação linear à consulta e à galeria.

3.6 A garantia de consistência de modelo

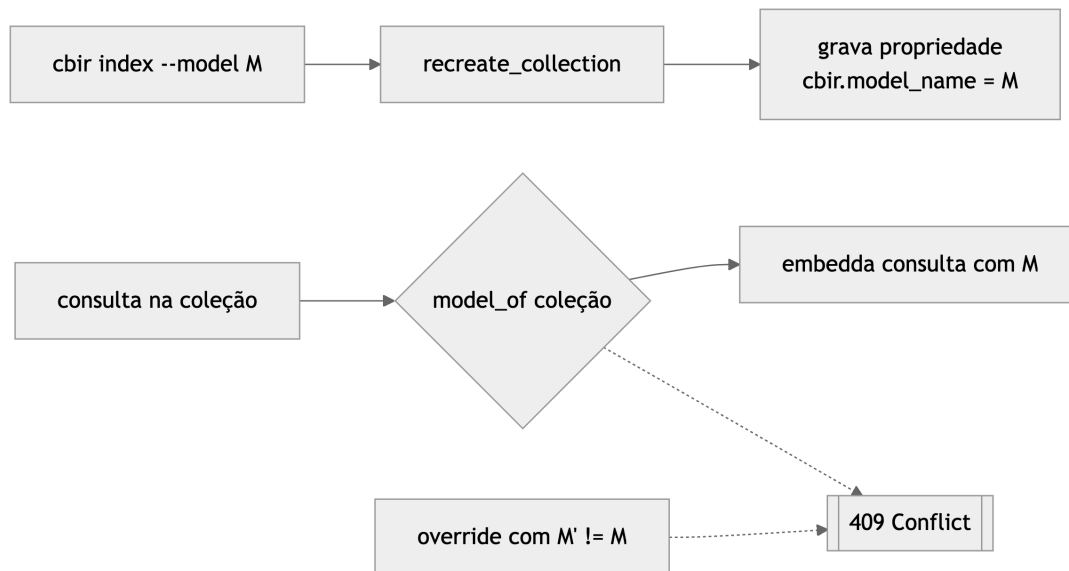


Figura 6: A coleção grava o modelo que a construiu; a consulta é sempre *embeddada* com ele, e qualquer *override* incompatível é recusado.

Um trecho do serviço que implementa a garantia:

```

collection_model = self.model_for_collection(collection)
chosen = model_name or collection_model
if collection_model and model_name and model_name != collection_model:
    raise ModelMismatchError(
        f"Collection {collection!r} was built with {collection_model!r}, "
        f"but the query requested {model_name!r}. "
        "Embeddings would be incomparable."
    )

```

3.7 Resolução de *device*

O sistema escolhe o melhor acelerador disponível e degrada com elegância (Figura 7), garantindo que o *mesmo* comando rode em qualquer máquina: CUDA, depois GPU Apple (MPS/Metal), depois CPU *multi-thread*.

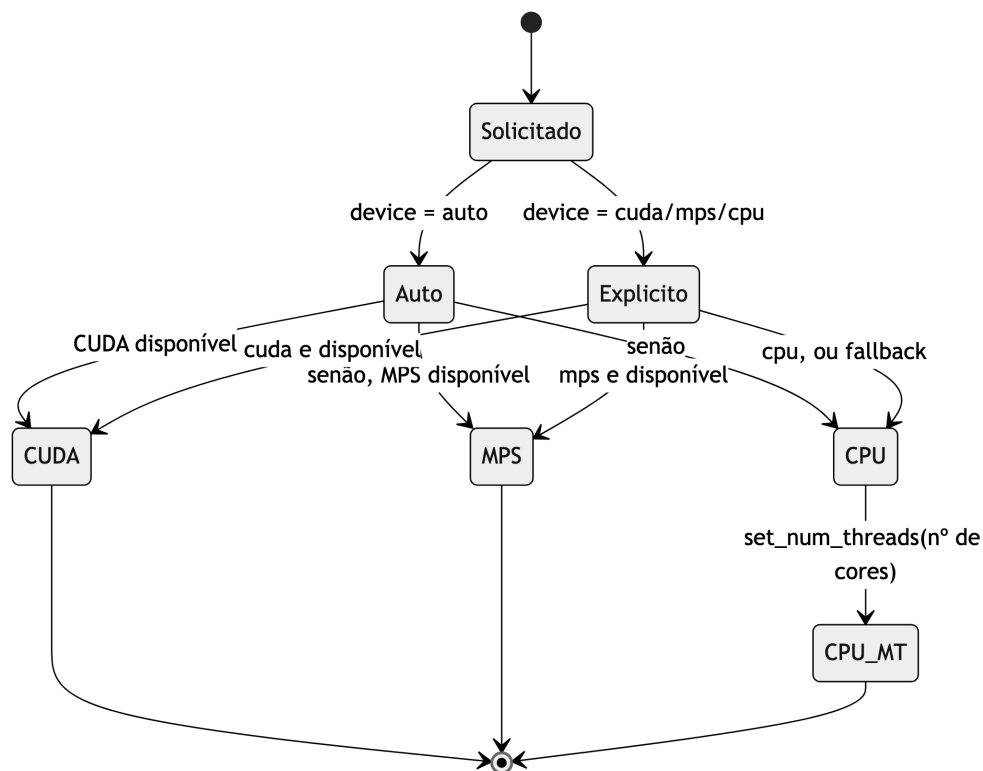


Figura 7: Máquina de estados da resolução de *device* (auto → CUDA → MPS → CPU).

3.8 Implantação (Docker Compose)

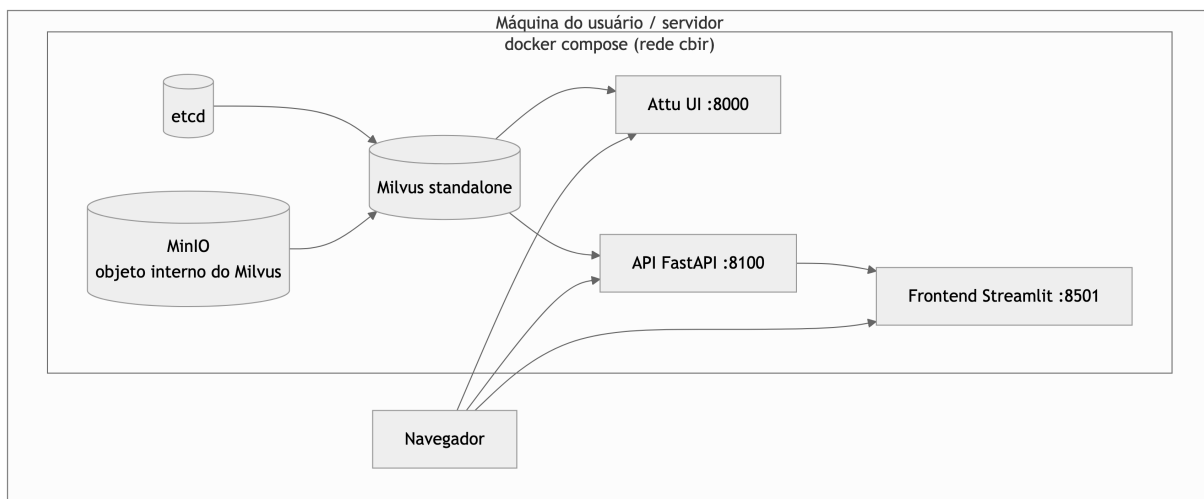


Figura 8: Diagrama de implantação. O perfil padrão sobe apenas o Milvus; o perfil *app* adiciona API e *frontend*.

3.9 Sobre o código

Aspecto	Decisão
Linguagem	Python 3.13, gerenciada por <code>uv</code>
Contratos de dados	Pydantic v2 (validação nas fronteiras entre camadas)
Extração	OpenCLIP; recorte em <i>runtime</i> ; L2-normalização
Banco vetorial	Milvus <i>standalone</i> (índice FLAT, métrica cosseno)
Projeção	scikit-learn PCA (2D/3D), com degradação graciosa
API	FastAPI (endpoints finos sobre o serviço)
<i>Frontend</i>	Streamlit + Plotly (fala apenas com a API)
<i>Device</i>	auto : CUDA → Apple MPS → CPU <i>multi-thread</i>
Observabilidade	<code>logging stdlib</code> com <i>wide events</i>
Qualidade	<code>ruff</code> (lint), <code>mypy</code> (tipos), <code>pytest</code> (testes)

Tabela 4: Principais decisões de implementação.

Estratégia de comentários: *docstrings* explicam a *intenção* e o *porquê* de cada módulo/função; comentários em linha marcam decisões não óbvias (ex.: por que negar similaridade negativa no voto, por que PCA e não UMAP). Código óbvio não é comentado.

Estratégia de testes: o *backend* é puro e testado sem Milvus/torch (projeção, KNN, *manifest*, modelos); a API é testada com um serviço falso, verificando inclusive a garantia de consistência de modelo (409). São 30 testes, executados com:

```
uv run ruff check cbir/ tests/ # lint
uv run mypy cbir/ # tipos
uv run pytest # testes
```

4 Manual de Utilização

O manual cobre os dois tipos de usuário visados: quem quer **rodar a demo** rapidamente e quem quer **indexar dados próprios**.

4.1 Instalação

```
uv sync # instala dependencias
docker compose up -d # sobe Milvus (etcd + minio + milvus + Attu)
# aguarde o Milvus ficar "healthy" (docker compose ps)
```

4.2 Tarefa A — Rodar a demonstração (sem GPU, sem baixar modelo)

```
uv run cbir seed --collection cbir_sample \
  --parquet cbir/sample_data/embeddings.parquet
uv run cbir api # terminal 1 -API em :8100
uv run cbir app # terminal 2 -frontend em :8501
# abra http://localhost:8501, escolha "cbir_sample" e envie um recorte
```

Exceções e problemas comuns:

- “API not reachable” — confirme que `cbir api` está rodando e a porta 8100 está livre.
- *Gráfico vazio* — rode `cbir seed` antes de abrir o app; a coleção precisa existir.

4.3 Tarefa B — Indexar dados próprios

```
uv run cbir index --manifest <caminho> --collection <nome> \
  --model openclip-vit-b-32 --device auto
# (opcional) snapshot reproduzível:
uv run cbir export --collection <nome> --model openclip-vit-b-32
```

Exceções e problemas comuns:

- “No records selected” — revise `--split` / `--benchmark-only` / `--per-class`.
- *Consulta recusada com 409* — a coleção foi construída com outro modelo; use o modelo da coleção.

4.4 Referência de comandos e endpoints

Comando	O que faz
<code>cbir sample</code>	Constrói o <i>dataset</i> de amostra comitável (recortes + <i>manifest</i>)
<code>cbir index</code>	Extraí <i>embeddings</i> de um <i>manifest</i> e indexa no Milvus
<code>cbir export</code>	Exporta os <i>embeddings</i> de uma coleção para um cache Parquet
<code>cbir seed</code>	Reconstrói uma coleção a partir de um cache Parquet (sem modelo)
<code>cbir api</code>	Sobe o serviço FastAPI
<code>cbir app</code>	Sobe o <i>frontend</i> Streamlit

Tabela 5: Comandos da CLI.

Método	Rota	Função
GET	<code>/health</code>	Verificação de vida
GET	<code>/models</code>	Modelos de <i>embedding</i> disponíveis
GET	<code>/collections</code>	Coleções indexadas + modelo + contagem
GET	<code>/collections/{n}/project</code>	Coordenadas PCA da galeria (2D/3D)
POST	<code>/collections/{n}/query</code>	Imagem → vizinhos + predição KNN + coords
GET	<code>/crop</code>	Serve um recorte por caminho de <i>manifest</i>

Tabela 6: Endpoints da API.

5 Verificação

O sistema foi validado de ponta a ponta na máquina de desenvolvimento (Apple Silicon, *device mps*).

Verificação	Resultado
Indexação da amostra (160 recortes, 4 classes)	160 itens em ≈ 29 s (MPS)
Projeção PCA 2D/3D da galeria	160 pontos; transform exato
Consulta ponta a ponta (upload → busca → KNN → projeção)	Predição coerente com confiança
Garantia de consistência de modelo	Recusa (409) confirmada em teste
<code>ruff</code> / <code>mypy</code> / <code>pytest</code>	Limpo / limpo / 30 testes passando
Reconstrução via cache (<code>seed</code>)	Coleção recriada em ≈ 3 s sem GPU

Tabela 7: Resultados da verificação de ponta a ponta.

6 Conclusão e Trabalhos Futuros

O programa entrega um sistema CBIR funcional e reproduzível, com três camadas bem separadas, uma garantia explícita de consistência de modelo, e uma interface visual que torna concreta a pergunta central da pesquisa: *uma imagem de classe A se posiciona perto de outras da classe A e longe das demais?* Como trabalhos futuros, destacam-se: (i) a comparação sistemática entre modelos de *embedding*; (ii) o uso de *embeddings* derivados de um detector treinado no domínio; e (iii) a avaliação quantitativa da confiabilidade dos limiares de similaridade para rotulagem automática, que é o objeto da dissertação.